JOBSHEET JAVA GUI – PART II
BE FE and CRUD

A. Learning Objectives

After completing this jobsheet, students are expected to be able to:

1. Understand the concept of Frontend (FE) and Backend (BE) separation in Java Swing-based desktop applications.
2. Create a program structure by separating business logic, data access, and display (UI).
3. Implement CRUD operations using Java
4. Using JTable as a data view component.
5. Using OOP (Class Entity, Class DAO, Class Service) to build a structured CRUD system.
6. Connect the input form in Java Swing to the backend using an event listener.
7. Manage data using ArrayList as temporary storage (database simulation

A. Theoretical Basis

1. FE–BE Architecture in Java Desktop Applications

Although desktop applications do not use HTTP requests like web applications, the concept of separation of FE and BE is still relevant for programs:

- more structured,
- easier to develop,
- Easy to fix when an error occurs,
- support the **principle of Single Responsibility** in OOP.

Frontend (FE):

The part of the user interface that is displayed via the Swing component (JFrame, JTextField, JButton, JTable, etc). Functions to display forms, tables, and accept user input.

Backend (BE):

The part that runs business logic and data management, including:

- Class Entity
- Class DAO (Data Access Object)
- Class Service

2. Class Entity

A class that represents a single data object. Example: Student Entity has the attribute:

- NIM
- Name
- Study Program

It is used as a data model that will move between FE and BE components.

3. Data Access Object (DAO)

A DAO is a class that is responsible for the entire process of data management. In this jobsheet, the DAO uses **ArrayList** as a temporary storage.

DAO Tasks:

- Save data → Create
- Retrieve data → Read
- Change → Update data
- Menghapus data → Delete

With DAOs, FE doesn't directly change the data—but rather BE processes everything.

4. Service Layer

Layers that bridge between:

- FE (buttons on Swing)

- DAO (data)

Service Works:

- Data validation
- Calling DAO methods
- Keep FE anonymous to data storage details

5. JTable to Display Data

JTable is used to display data in the form of a dynamic table. Usually paired with:

- **DefaultTableModel** or
- **AbstractTableModel**

Function:

- Displays all data from the DAO (Read)
- Repopulate tables whenever there is a change in data (Create/Update/Delete)

6. Event Handling on CRUD

Each button has an event listener:

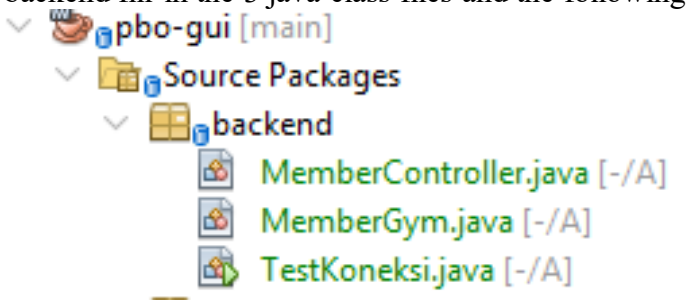
Knob	**Function**
Save	Calling a Service → DAO → New Data Added
Change	Calling a Service → DAO → Updated Data
Wipe	Delete data based on ID/NIM/Key
Reset	Cleaning the contents of the input components
Select Data from JTable	Fill out forms based on lines clicked

7. CRUD Workflow

1. User enters data in FE form
2. User presses the **Save/Update/Delete button**
3. FE calls the Service method
4. The service validates and calls the DAO
5. DAOs make data changes
6. The table in FE is updated to show the latest results

B. Exercise

1. Open Netbeans/VScode, create a new project and a new folder with the name of the backend fill in the 3 java class files and the following



2. On **MemberController.java** fill in the following code

```
package backend;

import java.sql.Connection;
import java.sql.PreparedStatement;

public class MemberController {

    public static boolean saveMember(MemberGym member) {
```

```
            try {
                Connection conn = TestConnection.getConnection();

                if (conn == null) {
                    System.out.println("Connection null! Can't save.");
                    return false;
                }

                sql string = "INSERT INTO member_gym (name, age, package)
    VALUES(?, ?, ?)";
                PreparedStatement stmt = conn.prepareStatement(sql);

                stmt.setString(1, member.getName());
                stmt.setInt(2, member.getAge());
                stmt.setString(3, member.getPackage());

                stmt.executeUpdate();

                stmt.close();
                conn.close();

                return true;

            } catch (Exception e) {
                System.out.println("Failed to save: " + e.getMessage());
                return false;
            }
        }
}
```

3. In the **MemberGym.java**, enter the following code

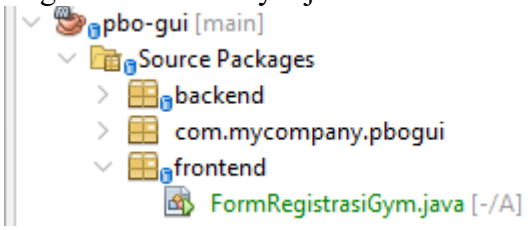
```
    package backend;

    This is a model;

    public class MemberGym {

        private String name;
        private int age;
        private String package;
        public MemberGym(Name string, int age, package string) {
            this.name = name;
            this.age = age;
            this.package = package;
        }
        public String getName() {
            return of name;
        }

        public int getUsia() {
            return of age;
        }

        public String getPackage() {
            return package;
        }
        public void setName(Name string) {
```

```
            this.name = name;
        }

        public void setAge(int age) {
            this.age = age;
        }

        public void setPackage(String package) {
            this.package = package;
        }
}
```

4. Make sure **TestKoneksi.java** is running properly and is connected to the database in your MySql, add this code to the file

```
package backend;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class TestKoneksi {

    public static void main(String[] args) {
        Connection conn = getConnection();

        if (conn != null) {
            System.out.println(x: "Koneksi BERHASIL!");
        } else {
            System.out.println(x: "Koneksi GAGAL!");
        }
    }

    public static Connection getConnection() {
        String url = "jdbc:mysql://localhost:3306/db_gym";
        String user = "root";
        String pass = "";

        try {
            Class.forName(className: "com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(url, user, password: pass);

        } catch (Exception e) {
            System.out.println("Koneksi gagal: " + e.getMessage());
            return null;
        }
    }
}
```

5. Please create a frontend folder, then create a java class file with the name RegistrasiMemberGym.java



6. Make sure you have imported the Controller and its Model in the header as in the following code advanced file. (pay attention to the writing of the folder call and the Java Class file)

```
package frontend;
```

```
import backend. MemberGym;
import backend. MemberController;
import javax.swing.*;
import java.awt.*;

public class FormRegistrasiGym extends JFrame {

    private JTextField txtName;
    private JTextField txtAge;
    private JComboBox<String> cmbPackage;
    private J.B. Scott;

    public FormRegistrationGym() {
        setTitle("Gym Member Registration Form");
        setSize(600, 400);                    ! Made wider & taller
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        initComponents();
    }

    private void initComponents() {

        // =========================
        // TITLE PANEL
        // =========================
        JLabel title = new JLabel("GYM REGISTRATION FORM",
SwingConstants.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setBorder(BorderFactory.createEmptyBorder(20, 10, 20,
10));
        add(title, BorderLayout.NORTH);


        // =========================
        // FORM PANEL
        // =========================
        JPanel formPanel = new JPanel(new GridBagLayout());
        formPanel.setBorder(BorderFactory.createEmptyBorder(20, 40,
20, 40));  Padding
        GridBagConstraints gbc = new GridBagConstraints();

        gbc.insets = new Insets(10, 10, 10, 10);   Spacing between
components
        gbc.fill = GridBagConstraints.HORIZONTAL;

        JLabel lblName = new JLabel("Full Name:");
        JLabel lblAge = new JLabel("Age:");
        JLabel lblPackage = new JLabel("Gym Package:");

        txtName = new JTextField(20);
        txtAge = new JTextField(20);

        String[] packageList = {"Monthly", "Yearly", "VIP"};
        cmbPackage = new JComboBox<>(packageList);

        btnSave = new JButton("Save");
        btnSave.setFont(new Font("Arial", Font.BOLD, 14));
```

```
            btnSave.addActionListener(e -> saveData());


            ===== ROW 1 =====
            gbc.gridx = 0; gbc.gridy = 0;
            formPanel.add(lblName, gbc);

            gbc.gridx = 1;
            formPanel.add(txtName, gbc);

            ===== ROW 2 =====
            gbc.gridx = 0; gbc.gridy = 1;
            formPanel.add(lblAge, gbc);

            gbc.gridx = 1;
            formPanel.add(txtAge, gbc);

            ===== ROW 3 =====
            gbc.gridx = 0; gbc.gridy = 2;
            formPanel.add(lblPackage, gbc);

            gbc.gridx = 1;
            formPanel.add(cmbPackage, gbc);

            ===== BUTTON =====
            gbc.gridx = 0; gbc.gridy = 3;
            gbc.gridwidth = 2;
            gbc.fill = GridBagConstraints.CENTER;
            formPanel.add(btnSave, gbc);

            add(formPanel, BorderLayout.CENTER);
        }

        private void saveData() {
            try {
                Name string = txtName.getText();
                int age = Integer.parseInt(txtAge.getText());
                Package string = cmbPackage.getSelectedItem().toString();

                MemberGym member = new MemberGym(name, age, plan);
                boolean successful = MemberController.saveMember(member);

                if (successful) {
                    JOptionPane.showMessageDialog(this, "Data saved
successfully!");
                } else {
                    JOptionPane.showMessageDialog(this, "Failed to save
data!");
                }

            } catch (Exception ex) {
                JOptionPane.showMessageDialog(
                        this,
                        "Error: " + ex.getMessage(),
                        "Input Error",
                        JOptionPane.ERROR_MESSAGE
                );
            }
```

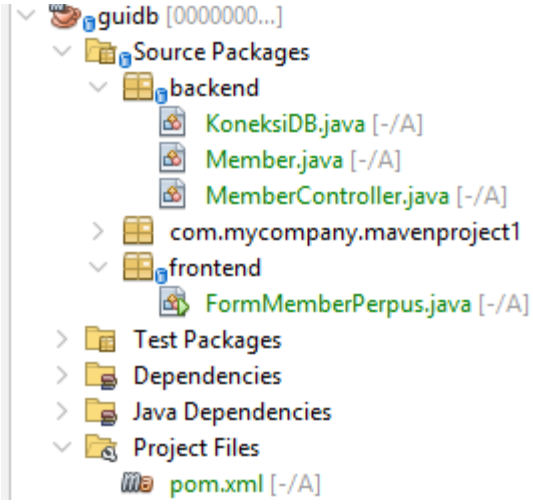
```
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> new
    FormRegistrasiGym().setVisible(true));
        }
}
```

7. Then right-click on the FormRegistrasiGym file and click run file, what is the output of the program?
8. What does backend (model and Controller) and frontend (view) mean, what's the difference? Explain!
9. If it still has an error or fails to save then pay attention to replace the pom.xml with the following code. Make sure the pom.xml has been conditioned by the JDBC Driver to be accessible or replace the following code on pom.xml

```
    <?xml version="1.0" encoding="UTF-8"?>
    <project xmlns="http://maven.apache.org/POM/4.0.0"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             http://maven.apache.org/xsd/maven-4.0.0.xsd">

        <modelVersion>4.0.0</modelVersion>

        <groupId>com.mycompany</groupId>
        <artifactId>pbogui</artifactId>
        <version>1.0-SNAPSHOT</version>
        <packaging>jar</packaging>

        <properties>
            <project.build.sourceEncoding>UTF-
    8</project.build.sourceEncoding>
            <maven.compiler.source>20</maven.compiler.source>
            <maven.compiler.target>20</maven.compiler.target>
            <exec.mainClass>com.mycompany.ghj.Ghj</exec.mainClass>
        </properties>

        <dependencies>
            <!-- MySQL JDBC Driver -->
            <dependency>
                <groupId>com.mysql</groupId>
                <artifactId>mysql-connector-j</artifactId>
                <version>8.2.0</version>
            </dependency>
        </dependencies>
        <name>pbo-gui</name>
</project>
```

C. Latihan CRUD (Create, Read, Update, Delete)

1. Please create a new project with the name guidb, here is the arrangement of the program code



2. Open your MySQL, and create a database with the following db_perpus name or copy code on the SQL homepage.

```
CREATE DATABASE db_perpus;

USE db_perpus;

CREATE TABLE member (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(200) NOT NULL,
    no_hp VARCHAR(20) NOT NULL,
    jenis_member VARCHAR(50) NOT NULL,
    tanggal_daftar DATE NOT NULL DEFAULT (CURRENT_DATE)
);
```

3. The first step is to create backend and frontend folders, then make sure pom.xml is in accordance with the previous instructions.
4. On the backend folder there are KoneksiDB.java

```
package backend;

import java.sql.Connection;
import java.sql.DriverManager;

public class KoneksiDB {
    private static final String URL =
"jdbc:mysql://localhost:3306/db_perpus";
    private static final String USER = "root";
    private static final String PASS = "";

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER,
PASS);
        } catch (Exception e) {
            System.out.println("Connection failed: " +
e.getMessage());
            return null;
```

```
            }
        }
}
```

5. Then fill in this Member.java as a Model on the MVC concept.

```
package backend;

public class Member {

    private int id;
    private String name;
    private String addresses;
    private String noHp;
    private String typeMember;

    public Member(int id, Name string, Address string, String
noHp, String typeMember) {
        this.id = id;
        this.name = name;
        this.address = address;
        this.noHp = noHp;
        this.typeMember = typeMember;
    }

    public Member(Name string, Address string, Hp noString,
Member-type string) {
        this.name = name;
        this.address = address;
        this.noHp = noHp;
        this.typeMember = typeMember;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getAddress() { return address; }
    public String getNoHp() { return noHp; }
    public String getTypeMember() { return typeMember; }

    public void setName(String name) { this.name = name; }
    public void setAddress(Address string) { this.address =
address; }
    public void setNoHp(String noHp) { this.noHp = noHp; }
    public void setTypeMember(String typeMember) {
this.typeMember = typeMember; }
}
```

6. Finally, in the backend folder, enter the following MemberController.java code as the Controller in the MVC concept.

```
package backend;

import java.sql.*;
import java.util.ArrayList;
```

```
public class MemberController {

    INSERT
    public static boolean add(Member m) {
        sql string = "INSERT INTO members (name, address,
no_hp, jenis_member) VALUES(?, ?, ?, ?)";

        try (Connection conn = ConnectionDB.getConnection();
PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, m.getName());
            ps.setString(2, m.getAddress());
            ps.setString(3, m.getNoHp());
            ps.setString(4, m.getTypeMember());

            ps.executeUpdate();
            return true;

        } catch (Exception e) {
            System.out.println("Error added: " +
e.getMessage());
            return false;
        }
    }

    // SELECT ALL
    public static ArrayList<Member> getAll() {
        ArrayList<Member> list = new ArrayList<>();
        sql string = "SELECT * FROM members";

        try (Connection conn = ConnectionDB.getConnection();
Statement st = conn.createStatement(); ResultSet rs =
st.executeQuery(sql)) {

            while (rs.next()) {
                list.add(new Member(
                    rs.getInt("id"),
                    rs.getString("name"),
                    rs.getString("address"),
                    rs.getString("no_hp"),
                    rs.getString("jenis_member")
                ));
            }

        } catch (Exception e) {
            System.out.println("Error displayed: " +
e.getMessage());
        }
        return list;
    }

    UPDATE
    public static boolean update(Member m) {
```

```
            sql string="UPDATE member SET name=?, address=?,
      no_hp=?, jenis_member=? WHERE id=?";

            try (Connection conn = ConnectionDB.getConnection();
      PreparedStatement ps = conn.prepareStatement(sql)) {

                ps.setString(1, m.getName());
                ps.setString(2, m.getAddress());
                ps.setString(3, m.getNoHp());
                ps.setString(4, m.getTypeMember());
                ps.setInt(5, m.getId());

                ps.executeUpdate();
                return true;

            } catch (Exception e) {
                System.out.println("Error update: " +
      e.getMessage());
                return false;
            }
        }

        DELETE
        public static boolean delete(int id) {
            sql string = "DELETE FROM member WHERE id=?";

            try (Connection conn = ConnectionDB.getConnection();
      PreparedStatement ps = conn.prepareStatement(sql)) {

                ps.setInt(1, id);
                ps.executeUpdate();
                return true;

            } catch (Exception e) {
                System.out.println("Error delete: " +
      e.getMessage());
                return false;
            }
        }
}
```

7. Next, in the frontend folder, create a java class file with the name **FormMemberPerpus**

```
      package frontend;

      import backend.*;
      import javax.swing.*;
      import javax.swing.table.DefaultTableModel;
      import java.awt.*;
      import java.util.ArrayList;

      public class FormMemberPerpus extends JFrame {

          private JTextField txtName, txtAddress, txtHp;
          private JComboBox<String> cmbType;
```

```
    private JTable table;
    private DefaultTableModel model;

    private int idSelected = -1;

    public FormMemberPerpus() {
        setTitle("Library Member Form");
        setSize(700, 450);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        initComponents();
        loadData();
    }

    private void initComponents() {

        JLabel lblName = new JLabel("Name");
        JLabel lblAddress = new JLabel("Address");
        JLabel lblHp = new JLabel("No HP");
        JLabel lblType = new JLabel("Member Type");

        txtName = new JTextField();
        txtAddress = new JTextField();
        txtHp = new JTextField();

        cmbType = new JComboBox<>(new String[]{"Regular",
"Premium", "VIP"});

        JButton btnSave = new JButton("Save");
        JButton btnUpdate = new JButton("Update");
        JButton btnDelete = new JButton("Delete");
        JButton btnReset = new JButton("Reset");

        btnSave.addActionListener(e -> save());
        btnUpdate.addActionListener(e -> update());
        btnDelete.addActionListener(e -> delete());
        btnReset.addActionListener(e -> resetForm());

        model = new DefaultTableModel(new String[]{"ID", "Name",
"Address", "No HP", "Type"}, 0);
        table = new JTable(model);

        table.getSelectionModel().addListSelectionListener(e ->
{
            if (!e.getValueIsAdjusting() &&
table.getSelectedRow() != -1) {
                isiForm();
            }
        });

        JPanel form = new JPanel(new GridLayout(5, 2, 10, 10));
        form.add(lblName); form.add(txtName);
        form.add(lblAddress); form.add(txtAddress);
```

```
            form.add(lblHp); form.add(txtHp);
            form.add(lblType); form.add(cmbType);

            JPanel button = new JPanel();
            button.add(btnSave);
            button.add(btnUpdate);
            button.add(btnDelete);
            button.add(btnReset);

            add(form, BorderLayout.NORTH);
            add(new JScrollPane(table), BorderLayout.CENTER);
            add(button, BorderLayout.SOUTH);
        }

        LOAD DATA TO A TABLE
        private void loadData() {
            model.setRowCount(0);
            ArrayList<Member> list = MemberController.getAll();
            for (Member m : list) {
                model.addRow(new Object[]{
                    m.getId(), m.getName(), m.getAddress(),
    m.getNoHp(), m.getMemberType()
                });
            }
        }

        SAVE
        private void store() {
            Member m = new Member(
                txtName.getText(),
                txtAddress.getText(),
                txtHp.getText(),
                cmbType.getSelectedItem().toString()
            );

            if (MemberController.add(m)) {
                JOptionPane.showMessageDialog(this, "Successfully
    saved");
                loadData();
                resetForm();
            }
        }

        UPDATE
        private void update() {
            if (idSelected == -1) {
                JOptionPane.showMessageDialog(this, "Select data
    first");
                return;
            }

            Member m = new Member(idSelected,
                txtName.getText(),
                txtAddress.getText(),
```

```
                    txtHp.getText(),
                    cmbType.getSelectedItem().toString()
            );

            if (MemberController.update(m)) {
                JOptionPane.showMessageDialog(this, "Successfully
    updated");
                loadData();
                resetForm();
            }
        }

        WIPE
        private void delete() {
            if (idSelected == -1) {
                JOptionPane.showMessageDialog(this, "Select data
    first");
                return;
            }

            if (MemberController.delete(idSelected)) {
                JOptionPane.showMessageDialog(this, "Successfully
    deleted");
                loadData();
                resetForm();
            }
        }

        RETRIEVE DATA TO FORM
        private void isiForm() {
            int row = table.getSelectedRow();
            idSelected = Integer.parseInt(model.getValueAt(row,
    0).toString());
            txtName.setText(model.getValueAt(row, 1).toString());
            txtAddress.setText(model.getValueAt(row, 2).toString());
            txtHp.setText(model.getValueAt(row, 3).toString());
            cmbType.setSelectedItem(model.getValueAt(row,
    4).toString());
        }

        // RESET FORM
        private void resetForm() {
            txtName.setText("");
            txtAddress.setText("");
            txtHp.setText("");
            cmbType.setSelectedIndex(0);
            idSelected = -1;
            table.clearSelection();
        }
        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> new
    FormMemberPerpus().setVisible(true));
        }
}
```

8. Right click run file, what is the output, try all of Create, Read, Update, Delete and screenshot the results of all the experiments and conditions on its MySQL database.

D. Assignment

1. Based on the previous Library Project, provide additional input validation, add validation that the name must not be blank, the phone number must be a number, and the address must be at least 10 characters. Display error messages using JOptionPane.
2. Add three new columns to the Database db_perpus> member table

   - `Email`
   - `tanggal_daftar`
   - `Status` (On/Inactive)

   Student Assignments:

   1. Update the database.
   2. Change the model class (member.java).
   3. Changing the backend code (insert, update, delete).
   4. Added an input field in the frontend.

---- GOOD LUCK-----